

Enhancing Facial Recognition Accuracy Through Image Processing Techniques

Rowida Morsy

Faculty of Computer Science
October University for Modern,
Sciences and Arts (MSA),
6th of October City, Giza, Egypt
rowida.morsy@msa.edu.eg,
ORCID ID:0009-0001-2903-6563

Tamer M. Nassef

Faculty of Computer Science
October University for Modern,
Sciences and Arts (MSA),
6th of October City, Giza, Egypt
tnassef@msa.edu.eg, tamer@ieee.org,
ORCID ID: 0000-0001-5301-6848

Abstract—Facial recognition systems face significant challenges when dealing with low-quality or noise-affected images, which reduces model accuracy and impacts system reliability. This research aims to improve the quality of these images by applying a range of preprocessing techniques to grayscale images to prepare them as effective input for facial recognition systems. The methodology includes using a median filter to reduce noise, histogram equalization to improve illumination, contrast stretching to increase contrast, sharpening to enhance detail, and gamma correction to improve brightness. The results showed a significant improvement in image clarity, light distribution, and noise reduction for most images, while some images did not improve significantly due to limited original information. This study underscores the importance of preprocessing in enhancing the efficiency of facial recognition systems, especially when using low-quality grayscale images.

Keywords: Face recognition, CNN (convolutional neural network), image processing, low-resolution images.

I. INTRODUCTION

Facial recognition technology is one of the most prominent applications of artificial intelligence and image processing in the modern era, due to its widespread use in diverse fields such as security and surveillance, digital identity, smartphones, and entertainment applications [1] [2]. The accuracy of any facial recognition system depends on the quality of the images used as input. Low-quality or blurry images directly affect system performance and may lead to inaccurate results or an inability to accurately recognize faces [3]. Realistic images often face several challenges, including noise caused by unsuitable lighting conditions, blurred facial details due to camera quality or movement during shooting, varying levels of light and contrast between different images, and the effects of background changes. All these factors render original images unsuitable for use in facial recognition systems. Therefore, this research aims to apply a set of image enhancement techniques to make them clearer and more reliable as inputs for intelligent systems.

The steps used in this research include noise removal using a median filter to eliminate random noise and optimizing light distribution through histogram equalization to ensure balanced brightness levels [5]. Contrast extension was used to widen the contrast between shadows and highlights, image sharpening to highlight edges and fine facial details, and finally, gamma correction to adjust overall brightness and improve the final image quality. These techniques help improve image clarity, reduce noise, and reveal fine details, thus contributing to increased accuracy in facial recognition systems [4]. The dataset used was carefully selected, containing images with various problems such as noise, lack of detail, and varying lighting levels, making them unsuitable for direct use in facial recognition systems. After image processing, the images were divided into separate folders for each individual, with each folder containing all images of that person. This organization facilitates future processing, improves the accuracy of system training and testing, and ensures the data is organized and ready for practical use.

This research focuses on improving the quality of grayscale images used in facial recognition systems by manually applying a series of preprocessing techniques using Python. These techniques include noise filtering, light distribution optimization, contrast enhancement, image sharpening, and gamma correction. Images will also be organized into separate folders for each person to facilitate processing.

The expected results indicate that these techniques will improve image clarity, reduce noise, and increase data quality before feeding it into a convolutional neural network (CNN) for higher facial recognition accuracy [6],[11].

II. PROPOSED SOLUTION

A. database preparation

The proposed solution began with a database setup phase, a crucial step to ensure the model's effectiveness and the accuracy of its results. A dataset from Kaggle containing 10

male and female individuals was used, totaling 1005 grayscale images. The number of photos per person ranges from 98 to 101, and the photo dimensions vary from 121×121 to 219×219 pixels, as shown in the following table:

TABLE I: Number of photos and example resolutions

Person	Number of Photos	Example Resolution (pixels)
Abir	101	135×135
Tarup	101	208×208
Rupak	101	151×151
Nipa	101	186×186
Hemel	101	219×219
Shepon	98	140×140
Sabiqul	101	121×121
Fahim	101	187×187
Tamim	100	151×151
Tama	100	196×196

The images were captured in an uncontrolled environment with varying lighting and angles, making large-scale discrimination as challenging as real-world requirements. To organize and structure the images, each person's images were placed in a separate section, divided into 80% enhanced images for training and 20% original images for testing, within the training and validation analysis phases. The quality of the damaged images was checked, and the original model of the results was reconstructed. Each stage of data preparation, including the number of images, their division, and adjustments, was documented as a structured system, resulting in an efficient model with accurate results, stable performance, and dynamic differences.



Fig. 1: Examples of original, unprocessed images .

1) Applying the Median Filter in Image Processing

The median filter was used as the first step in the image enhancement process. This filter aims to remove salt and pepper noise from grayscale images without affecting edges and important details

$$I'(x, y) = \text{median} \{I(s, t) \mid (s, t) \in \text{window}(x, y)\}$$

Where $\text{window}(x, y)$ represents a square of pixels around the pixel (x, y) of size $k \times k$ (for example, 3×3).

The filter sorts the pixel values within this square and selects the middle value as the new pixel value.



Fig. 2: After apply median filter

2) Contrast stretching

During the image enhancement phase, contrast stretching was applied to enhance the clarity of the grayscale image and

increase the contrast between light and dark areas [8]. This technique aims to redistribute pixel values so that they span the entire range of available gray levels (from 0 to 255), thereby highlighting subtle details hidden within the image. Contrast stretching is mathematically performed on each pixel p in the image according to the following equation:

$$p_{\text{new}} = \frac{p - p_{\min}}{p_{\max} - p_{\min} + \epsilon} \times 255 \quad (1)$$

where p_{new} represents the pixel value after stretching, p the original pixel value, and $p_{\min}$ and $p_{\max}$ are the minimum and maximum pixel values in the image, respectively. The constant $\epsilon = 1 \times 10^{-5}$ is added to avoid division by zero in case the minimum and maximum values are equal. As a result of this procedure, the differences between pixel values become more pronounced, and the contrast range expands, improving image clarity and making fine details more visible.

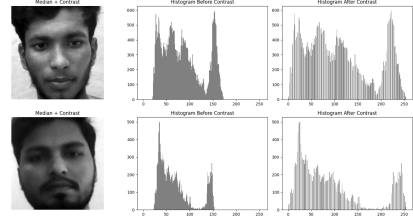


Fig. 3: After apply contrast stretching and median filter

3) Histogram Equalization

During the image enhancement phase, histogram equalization was used to improve the contrast of the grayscale image and enhance detail. The goal of this process is to redistribute pixel values so that they are evenly distributed across the entire range of available gray levels, making light and dark shadows more distinct and balanced [9].

Histogram Calculation and Probability Function

First, we calculate the histogram of the image, which is the number of pixels that each specific grayscale value (i from 0 to 255 in the case of an 8-bit image) . if $h(i)$ represents the number of pixels with the value i , we calculate the probability function PDF (Probability Density Function) for each value as follows:

$$\text{PDF}(i) = \frac{h(i)}{H \times W} \quad (2)$$

Where H and W are the height and width of the image. This function gives the probability of each gray level appearing in the image.

Calculating the Cumulative Distribution Function

Next, we calculate the Cumulative Distribution Function (CDF), which represents the sum of the probabilities of all values from 0 to i :

$$\text{CDF}(i) = \sum_{j=0}^i \text{PDF}(j) \quad (3)$$

The CDF function provides a cumulative chance of reaching each gray level and is used to reset pixel values [7].

Reset Pixel Values

The value of each pixel in the new image $I'(x, y)$ at the location (x, y) is calculated as follows:

$$I'(x, y) = \text{round}(\text{CDF}(I(x, y)) \times (L - 1)) \quad (4)$$

Where $I(x, y)$ is the original pixel value, and L is the number of gray levels (usually 256). The rounding operation (rounding) is important to convert the result of multiplying by $L - 1$ to the nearest integer that can be used as a pixel, since pixel values must be integers between 0 and 255.

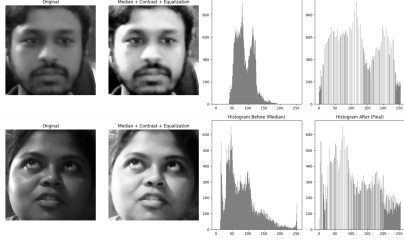


Fig. 4: After apply contrast stretching, median filter and Histogram Equalization

4) High-Boost Filtering

High-boost filtering is an image enhancement technique designed to sharpen fine details and accentuate edges while preserving the overall structure of the image [10]. This technique involves combining a low-frequency (blurred) version of the image with the original image in a way that amplifies the high-frequency signals responsible for the detail.

The process begins by calculating a blurred version of the original image, I , using a simple averaging filter. This is done by applying a two-dimensional kernel:

$$K = \frac{1}{n^2} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} \quad (5)$$

Where n represents the size of the kernel (e.g., 3×3). The blurred image is calculated using a convolution process:

$$I_{\text{blur}} = I * K \quad (6)$$

Next, the detail mask is calculated, which is the difference between the original image and the blurred image:

$$M = I - I_{\text{blur}} \quad (7)$$

This mask represents the high-frequency component in the image (edges and fine details). Then, the High-Boost formula is applied, which aims to enhance these details using the boosting factor k :

$$I_{\text{HB}} = I + k \cdot M \quad (8)$$

Substituting M , we get:

$$I_{\text{HB}} = (k + 1) I - k I_{\text{blur}} \quad (9)$$

The boosting factor k controls the degree of detail enhancement:

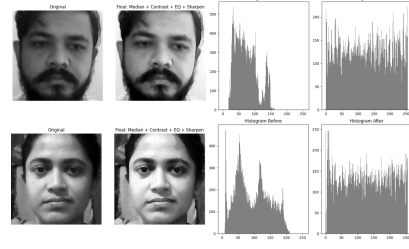


Fig. 5: After apply contrast stretching, median filter, histogram equalization, and High-Boost Filtering

- if $k = 1$: The process becomes similar to *Unsharp Masking*.
- if $k > 1$: The edges are enhanced more strongly.
- If $0 < k < 1$: The enhancement is mild and suitable for noise-sensitive images.

The resulting image values are then clipped within the standard range:

$$I_{\text{HB}} \leftarrow \text{clip}(I_{\text{HB}}, 0, 255) \quad (10)$$

this ensures the values remain within the image's grayscale range. This technique helps to bring out blurry edges and enhance weak details, making the resulting image clearer before use in later stages such as recognition or feature extraction. Furthermore, using it after steps like noise removal or histogram straightening achieves a balance between sharpness and lighting quality.

5) Gamma correction

Gamma correction aims to adjust the lighting in an image by applying a non-linear transformation to pixel values, thereby adjusting the clarity of details in dark or light areas. The process relies on altering the relationship between the actual light intensity and the digital values within the image, making it more akin to what the human eye perceives [9].

Basic Equation

The new value for each pixel is calculated using the relationship:

$$I_{\text{out}} = 255 \times \left(\frac{I}{255} \right)^\gamma \quad (11)$$

Where:

- I : Original pixel value,
- γ : Lighting control parameter,
- I_{out} : Corrected output value.

Processing Steps

a) 1 - Image Normalization

$$I_{\text{norm}} = \frac{I}{255} \quad (12)$$

b) 2 - Applying the Gamma Factor

$$I_\gamma = (I_{\text{norm}})^\gamma \quad (13)$$

c) 3 - Reverting the Image to its Original Gray Range

$$I_{\text{out}} = I_\gamma \times 255 \quad (14)$$

d) 4 - Adjusting Values within the Range $0 \rightarrow 255$

$$I_{\text{out}} \leftarrow \text{clip}(I_{\text{out}}, 0, 255) \quad (15)$$

Effect of Choosing a Value for γ

- If $\gamma < 1$: The image is brightened and detail in shadows is improved.
- If $\gamma > 1$: The image is darkened.

This results in a slight brightening of the image and an increase in detail clarity without affecting the original lighting conditions.

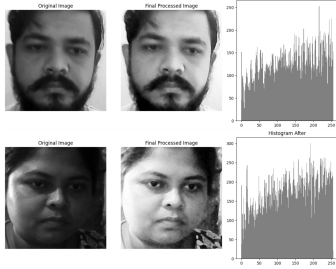


Fig. 6: After apply contrast stretching, median filter, histogram equalization, high-boost filtering, and Gamma correction

B. Model Training Using a Convolutional Neural Network (CNN)

1) Data Preparation

After completing the image enhancement and database organization phases, the dataset was divided into two sets:

- **Training Set:** containing the enhanced images.
- **Test Set:** containing the original, unenhanced images.

Before training, any corrupted images were removed PIL to ensure data quality and model stability. The random seed was set to 16 to minimize the effects of random distribution.

2) Data Initialization for Training

- In the **PyTorch** experiment: torchvision.transforms was used with rescale to $[0,1]$, batch size of 16, and the same random seed was applied to ensure the stability of the results.
- In the **Keras** experiment: ImageDataGenerator was used with rescale to $[0,1]$, batch size of 16, and the same random seed was applied to ensure the stability of the results.
- Normalize pixel values to the range $[0, 1]$
- Resize images to 128×128
- Select batch size (batch size) = 16
- Use multiple random seeds (seeds) to reduce the effect of randomization

3) CNN Model Structure

The model is designed as follows:

- 1) Two convolutional layers (Conv2D) with ReLU enabled
- 2) Two max pooling layers (MaxPooling2D) to reduce feature dimensions
- 3) Flatten data (Flatten)
- 4) One dense layer (Dense) with 128 units and ReLU enabled

- 5) Output layer (Dense) Number of units = Number of classes with Softmax enabled

4) Model Compilation and Training

- Optimizer: Adam
- Loss Function: CrossEntropy
- Number of Scans (Epochs): 5 for images before optimization, 10 for images after optimization

5) Model Evaluation

The model was evaluated on the test set after each trial using accuracy_score from the Scikit-learn library. To ensure the reliability of the results, the following were calculated:

- Average accuracy across all random seeds
- Standard deviation of model performance

The methodology combines data optimization, CNN model building, and iterative evaluation procedures to obtain a model capable of accurately recognizing grayscale faces for each individual within the dataset.

III. EXPERIMENTAL RESULTS

A. Data Set Overview

To illustrate the effect of image enhancement, samples of each individual were compared before and after processing, including noise removal, histogram normalization, contrast enhancement, and high-resolution filters. Figure 8 shows the improved clarity of features and fine details in the enhanced images, which strengthens the model's ability to distinguish between individuals more accurately and underscores the importance of the data enhancement phase in supporting the model's training and overall performance.



Fig. 7: A comparison of the original (left) and enhanced (right) image.

B. Random Seeds

Primarily, The random order was the primary factor, as the study used a specific set of dynamic elements (16, 26, 28, 50, and 15) for training and learning, ensuring the independence of the results from the order. All results obtained using seed 16 achieved the highest success rate after full image optimization

(97.1%) and 85% before optimization. This improvement indicates that image optimization techniques, including noise reduction, histogram normalization, and contrast signaling, support the model's ability to differentiate between categories.

The average accuracy across all results was $96.08\% \pm 1.08$, reflecting the model's reliability and the probabilistic nature of its results across diverse datasets. Therefore, these results are highly significant when combining random and selective data and represent multiple, reliable, and validated reviews for identification.

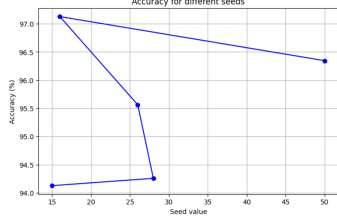


Fig. 8: Model accuracy across different random values.

C. Baseline CNN Model Performance

A simple convolutional neural network (CNN) was designed to test the baseline model's performance before any image enhancements or network architecture changes. The network consists of the following components:

- 1) **Two Convolutional Layers (Conv2D):** To extract key features from images such as edges and patterns, with ReLU enabled to add nonlinearity and enable the learning of complex representations.
- 2) **Two Max Pooling Layers (MaxPooling2D):** To reduce the dimensions of features and speed up training, while maintaining partial translations of features.
- 3) **Flatten Layer:** Converting multidimensional outputs into a single vector for dense layers.
- 4) **Output Layer:** 128 units with ReLU for assembling features and creating a comprehensive image representation.

The model was evaluated on the test set after training, and performance metrics were calculated for each class, including:

- 1) **Precision:** The proportion of correct predictions for each class compared to all predictions for that class.
- 2) **Recall:** The model's ability to recognize all real-world samples for each class.
- 3) **F1-score:** A combination of precision and recall to provide a standardized measure of performance.
- 4) **Overall Accuracy:** The percentage of correct predictions across all categories.

These results represent the performance of the base model before any additional image enhancements or architectural modifications, and are used as a benchmark for comparing the impact of image enhancement techniques on network performance later on.

The classification table and loss curve illustrate the model's performance before image enhancement. Some classes, such as Hemel and Shipon, exhibited low accuracy due to poor

TABLE II: Classification Report before optimization

Category	Precision	Recall	F1-score	Support
Abir	1.00	1.00	1.00	51
Fahim	1.00	1.00	1.00	52
Hemel	1.00	0.02	0.04	53
Nipa	1.00	0.86	0.92	49
Rupak	1.00	1.00	1.00	31
Sabiquil	1.00	1.00	1.00	31
Shepon	0.34	1.00	0.50	30
Tama	1.00	1.00	1.00	30
Tamim	1.00	1.00	1.00	30
Tarup	1.00	1.00	1.00	31
Accuracy		0.85		388
Macro Avg	0.93	0.89	0.85	388
Weighted Avg	0.95	0.85	0.82	388

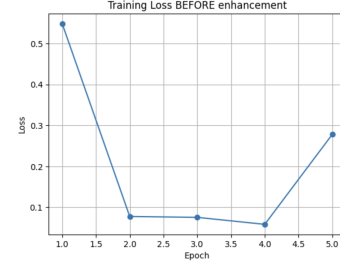


Fig. 9: Loss curve during training on unenhanced image.

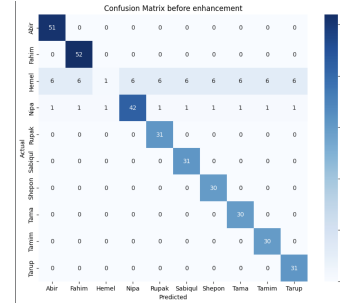


Fig. 10: Confusion matrix before image enhancement.

feature clarity in the original images, while others achieved higher accuracy. The loss curve shows a gradual decrease during training, indicating the model's ability to learn, but its difficulty in extracting subtle features, thus reducing overall performance on the test set.

TABLE III: Classification Report After optimization

Class	Precision	Recall	F1-score	Support
Abir	1.00	0.97	0.98	89
Fahim	1.00	1.00	1.00	88
Hemel	0.96	1.00	0.98	78
Nipa	1.00	1.00	1.00	82
Rupak	1.00	1.00	1.00	77
Sabiquil	1.00	1.00	1.00	77
Shepon	1.00	1.00	1.00	66
Tama	1.00	1.00	1.00	64
Tamim	1.00	1.00	1.00	72
Tarup	1.00	1.00	1.00	73
Accuracy		1.00		766
Macro Avg	1.00	1.00	1.00	766
Weighted Avg	1.00	1.00	1.00	766

Figure 8 illustrates the model's accuracy performance across different random values, showing stable results that reflect the model's reliability. Figure 9 presents the loss curve

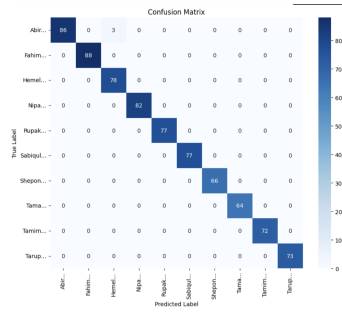


Fig. 11: Confusion matrix after image enhancement.

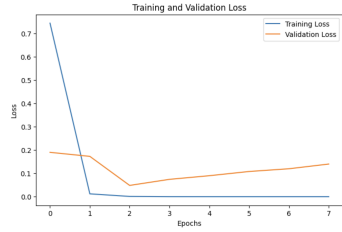


Fig. 12: Loss curve during training.

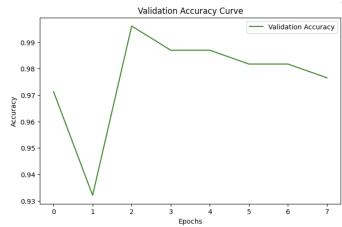


Fig. 13: Model accuracy on the validation set after image enhancement.

during model training on unenhanced images, demonstrating a gradual decrease in loss value as the training process progresses. Figure 10 shows the confusion matrix before image enhancement, revealing instances of confusion between some categories due to low image quality. After applying image enhancement techniques, Figure 11 shows the post-enhanced confusion matrix, exhibiting a significant increase in the number of correct predictions on the main diagonal and a decrease in errors between categories. Furthermore, Figure 12 illustrates the evolution of the loss value during training after image enhancement, indicating improved model learning. Finally, Figure 13 demonstrates the improved model accuracy on the validation set after applying image enhancement techniques.

IV. CONCLUSION AND DISCUSSION

The results of this study, consistent with research on hand gesture recognition and fruit defect detection, confirm that integrating preprocessing techniques with deep learning algorithms is crucial for improving model accuracy and generalizability. The EfficientNet-B0 model demonstrated superior gesture recognition, achieving 95.6% accuracy on the HGR1 dataset and 94% on the ASL dataset, surpassing the ResNet-18, VGG19, and AlexNet models [13]. Fruit defect detection systems achieved accuracy ranging from 89.96% to 95.23% using various preprocessing techniques, such as background

removal, histogram equalization, and edge detection [12]. The study concludes that image quality improvement is a critical element in convolutional neural network (CNN) systems, particularly when dealing with low-resolution images. Applying techniques such as noise reduction, contrast enhancement, and gamma correction led to a significant improvement in performance, with accuracy increasing from 85% before processing to 97.1%, averaging $96.08\% \pm 1.08$ across all datasets. This is attributed to the consistency of facial anatomical features and the availability of a sufficient number of images per category, which enhanced model generalization and reduced error rates. Comparisons across different domains showed that facial recognition was the most stable and easily applicable in real-world environments, followed by gesture recognition and then fruit defect detection, each with its own specific challenges. These results underscore that image enhancement is not an additional step but a fundamental component for developing more efficient and reliable models for future applications.

REFERENCES

- [1] E. A. Stephen, "Image preprocessing for face recognition," *Journal of Computing and Electronic Information Management*, vol. 19, no. 1, pp. 20, 2025.
- [2] M. Tripathi, "Facial image noise classification and denoising using neural network," *Sustainable Engineering and Innovation*, vol. 3, no. 2, pp. 102, 2021.
- [3] R. K. Shukla, A. Agarwal, and A. K. Malviya, "An Introduction of Face Recognition and Face Detection for Blurred and Noisy Images," *International Journal of Scientific Research in Computer Science and Engineering*, vol. 6, no. 3, pp. 39-43, 2018.
- [4] A. Lohar, K. Menaria, L. Choubisa, L. Deora, and L. Lohar, "Smart Attendance System Through Facial Recognition," 2023.
- [5] A. S. Lateef and M. Y. Kamil, "Face recognition-based automatic attendance system in a smart classroom," *Iraqi Journal for Electrical and Electronic Engineering*, vol. 20, no. 1, pp. 37-47, 2024.
- [6] A. Konisi, D. M. Laxmi, V. Sahithi, and B. A. Kumar, "Face Recognition Attendance System," *Journal of Nonlinear Analysis and Optimization*, vol. 14, no. 2, 2023.
- [7] A. Makandar, A. Patrot, and B. Halalli, "Color image analysis and contrast stretching using histogram equalization," *International Journal of Advanced Information Science and Technology*, vol. 3, no. 7, pp. 27, 2014.
- [8] D. R. Ningsih, "Improving retinal image quality using contrast stretching, histogram equalization, and CLAHE methods with median filters," *International Journal of Image, Graphics and Signal Processing*, vol. 14, no. 2, pp. 30, 2020.
- [9] M. A. Rahman, S. Liu, S. Lin, C. Wong, G. Jiang, and N. Kwok, "Image contrast enhancement for brightness preservation based on dynamic stretching," *International Journal of Image Processing (IJIP)*, vol. 9, no. 4, pp. 241, 2015.
- [10] N. J. Habeeb, "Medical image denoising with Wiener filter and high boost filtering," *Iraqi Journal of Science*, pp. 3123-3135, 2023.
- [11] M. O. Oloyede, G. P. Hancke, and H. C. Myburgh, "Improving face recognition systems using a new image enhancement technique, hybrid features and the convolutional neural network," *IEEE Access*, vol. 6, pp. 75181-75191, 2018.
- [12] Mouhebelidin, F. H. I., Mahmoud, N. a. M., Saad, M. N., Nassef, T. M. (2025). Smart Agriculture: Automated detection and classification of fruit diseases using CNN-Based image processing. International Conference on Electronic Engineering, 1-7. <https://doi.org/10.1109/iceem66692.2025.11225230>
- [13] Mohamed, M. Y., Mohamed, M. E., Saad, M. N., Nassef, T. M. (2025). Optimizing Sign Language Detection with Deep Learning and Preprocessing: A Comparative Study of CNN-Based Models. 1-7. <https://doi.org/10.1109/iceem66692.2025.11225228>